Assistant d'Onboarding RH Intelligent

Projet NLP — Master ISF 2026

RAG · Multi-Agents

Clément Le Berre · Martin Rullié · Audrey Ngoloho

Contexte & Objectif

Le problème

Un nouvel employé doit jongler entre :

- Documents RH éparpillés
- Annuaire employés à chercher
- Messages à envoyer manuellement
- Réunions à planifier

Notre solution

Un assistant conversationnel unique qui gère tout :

- Répond aux questions RH (RAG)
- Interroge la base employés
- Envoie des messages
- Planifie des réunions

Architecture Multi-Agents

Agent Principal (ReAct)

Module RAG
FAISS + Mistral
7 docs RH

DB Agent
SQLite
15 employés

Message Agent
Simulation
Slack

Meeting Agent
Calendrier
Simulé

Interface Streamlit · Sélecteur d'employé · Chat · Panneau Debug

Stack Technique

LLM

Mistral Small (temp. 0)

Embeddings

mistral-embed

Orchestration

LangGraph (ReAct)

Vectorstore

FAISS

BDD

SQLite

Interface

Streamlit

Module RAG — Base de connaissances

7 docs Markdown
mutuelle, horaires
télétravail, congés…

→

Chunking
500 chars
50 overlap

→

Embeddings
mistral-embed
vecteurs

→

FAISS Index
recherche
sémantique

→

Top 3 Chunks
contexte →
LLM → réponse

Points clés

- Réponses sourcées : les documents consultés sont cités dans chaque réponse
- Prompt contextuel : le LLM ne répond qu'à partir du contexte récupéré (pas d'hallucination)
- Index mis en cache : FAISS persisté sur disque pour les lancements suivants

Sub-Agents : Data, Messages & Réunions

DB Agent

Tools

```
get_employee(name)
get_manager(name)
search_employees(query)
```

SQLite · 15 employés
Recherche LIKE partielle

Message Agent

Tools

```
send_message(to, message)
```

Simulation Slack
Log : ID + timestamp + statut

Meeting Agent

Tools

```
check_availability(...)
schedule_meeting(...)
```

80% dispo / 20% conflit
Créneaux alternatifs

Orchestration en action

« Qui est mon manager ? Envoie-lui un message et planifie un point demain à 10h. »

1	`DB Agent → get_manager`	→	Manager : Bob Durand
2	`Message Agent → send_message`	→	Message envoyé (ID: msg-4827)
3	`Meeting Agent → check_availability`	→	Créneau disponible : 10h-10h30
4	`Meeting Agent → schedule_meeting`	→	Réunion planifiée (ID: mtg-1293)

Mode Debug / Trace

Panneau latéral activable

- Toggle dans la sidebar Streamlit
- Chaque étape : horodatage, agent, outil
- Arguments passés et résultat de chaque tool
- Sources RAG consultées (titres / IDs)
- Chaîne d'appels complète + durée
- Classe ExecutionTrace + ToolCallTrace

Exemple de trace

```
[14:32:01] DB Agent
  └─ get_manager("Jean Dupont")
     → Bob Durand

[14:32:03] Message Agent
  └─ send_message("Bob Durand", "...")
     → msg-4827 ✓

[14:32:04] RAG Module
  └─ query: "mutuelle"
     → Sources: mutuelle.md (chunks 1,3)

Durée totale: 3.2s
```

Difficultés rencontrées & Solutions

Orchestration multi-tools sur requêtes complexes

→ Pattern ReAct via LangGraph : l'agent raisonne étape par étape et chaîne les appels automatiquement

Prompt engineering du main agent

→ System prompt structuré listant les outils + cas d'usage + contraintes (français, refus hors-sujet, citation des sources)

Contexte employé dynamique

→ Injection du nom de l'employé connecté dans le system prompt à chaque requête via le sélecteur sidebar

Qualité du RAG (réponses non pertinentes)

→ Chunking optimisé (500/50 overlap) + instruction au LLM de ne répondre qu'à partir du contexte fourni

Conclusion & Perspectives

Ce qui fonctionne

- Orchestration fluide de requêtes multi-étapes
- RAG pertinent avec citations des sources
- Mode debug/trace complet et lisible
- Interface intuitive (chat + annuaire)
- Refus poli des questions hors-sujet

Pistes d'amélioration

- Amélioration RAG
- Intégration réelle Slack / Google Calendar
- Mémoire conversationnelle longue durée
- Ajout de modules (formation, e-learning)
- Déploiement cloud + authentification

Merci — Questions ?